

How `fmix` Decides a Parameter's Value

The layered-knob model, as of `e3cf2b54`

local_mixing working notes

2026-08-05

Abstract

A DB knob in `fmix` exists at up to three levels of *specificity* (base, mode, mode+geometry) and can be set from three *sources* (explicit command line, a named profile, the shipped defaults). Until 2026-08-05 those two dimensions were conflated and encoded in sentinel values, with the result that a shipped default could silently outrank an explicit flag — `--db-mode comp --s-db 20` ran at 12. This note states the replacement rule, shows where each step happens, and gives eleven worked examples taken verbatim from the binary. Everything here is DB-move knobs; the twist, thermostat and profile knobs are unchanged.

Contents

1	The two dimensions	2
2	The rule	2
2.1	Presets are deliberately not withheld the same way	2
3	Where each step happens	3
3.1	Step 1 — composing sources	3
3.2	Step 4 — resolving specificity	3
4	What the layers contain	3
4.1	What is NOT layered	3
5	Worked examples	4
6	Reading a run	5
7	Errors	5
8	Compatibility	5
9	Where to look	5

1 The two dimensions

Specificity — how narrowly a knob is scoped.

level	example	applies to
base	<code>--s-db</code>	every round
mode	<code>--s-db-comp</code>	COMP rounds
mode + geometry	<code>--s-db-comp-ctg</code>	COMP rounds that drew a contiguous window

Source — where a value came from.

source	example
explicit CLI	the user typed <code>--s-db 20</code>
preset	<code>--gss</code> (over <code>--phase-a</code>)
shipped default	what <code>fmix</code> does when told nothing

The old scheme had no representation for *source*. Overrides were sentinel-encoded — 0 meant “unset” for a `usize`, negative for an `f64` — so “the user asked for 12” and “12 is merely the default” were the same bit pattern. That works only while the shipped default *is* the sentinel. Commit `e45311ab` gave `s_db_comp` a default of 12 and `p_convex_comp` a default of 0.9, at which point both fired unconditionally, and any explicit `--s-db` / `--p-convex` was silently discarded in COMP rounds. Two arms of the measurement campaign were mis-run this way before it was noticed.

A second, quieter casualty: 0 is a *legitimate* value for some of these knobs. `--p-mingen-comp 0` is exactly what the GSS profile wants, and under a sentinel scheme it was inexpressible.

2 The rule

Sources are ranked, highest first:

explicit CLI > preset (<code>--gss</code> over <code>--phase-a</code>) > shipped default
--

Within a source, the more specific level wins at use time (mode+geometry → mode → base). And one exception, which is the entire reason the change exists:

A shipped MODE-level default is withheld when the corresponding BASE knob is given explicitly.

So `--db-mode comp --s-db 20` now runs COMP at 20. The justification is that a shipped default is not a statement about *this* run, and must not outrank one that is.

2.1 Presets are deliberately not withheld the same way

`--gss --s-db 15` moves MIX to 15 and leaves COMP at the profile’s 12/6. A named profile is a coherent unit whose mode-level choices are intentional — unlike a shipped default, it *is* a statement about this run. To move COMP as well, say `--s-db-comp`. This asymmetry is a judgement call and is reversible in two lines (`DbLayer::shipped`’s `base_given` treatment applied to the preset layer as well).

3 Where each step happens

Deciding *which source* a value came from is the CLI’s job, because only there does clap’s `ValueSource` distinguish a typed flag from a default. Deciding *which specificity level* applies is the mixer’s job, because only there is the live mode and the drawn geometry known.

step	where	what
1	<code>fmix.rs</code> , <code>DbLayer</code>	compose sources into one layer
2	<code>fmix.rs</code> , validation	reject knobs that cannot fire
3	<code>fmix.rs</code> , banner	print the settled table
4	<code>mix.rs</code> , <code>MixParams::db_knobs</code>	resolve specificity, per round

3.1 Step 1 — composing sources

`DbLayer` is eleven `Option` fields, one per knob. `None` means “this layer has no opinion”. Composition is `a.over(b)`: `a` wins wherever it has an opinion, `b` fills the rest. The whole resolution is one line:

```
let db = cli.over(preset).over(DbLayer::shipped(base_given));
```

`cli` maps a typed flag to `Some`. For the four base knobs that still carry a `default_value_t` this uses `given(name)`; for the seven overrides the CLI type is itself `Option`, so `Some` already means “the user asked”.

`--no-db-prefixes` folds in here as *explicitly false* rather than surviving as a parallel mechanism.

Presets contribute a layer rather than mutating `args`. That matters: under the old in-place mutation nothing downstream could tell “the user asked for 12” from “`--gss` set 12”.

3.2 Step 4 — resolving specificity

`MixParams::db_knobs(mode)` returns the five settled values for a mode. The four `Mixer::active_*` accessors are thin wrappers over it, so there is exactly one implementation of the fall-through.

One asymmetry worth knowing: **COMP contiguous falls back to COMP convex, not straight to the base**. A run that said `--s-db-comp 7` meant it for both geometries; only `--s-db-comp-ctg` separates them.

4 What the layers contain

`--gss` is deliberately silent on `p_mix`: the MIX/COMP balance is layer 2’s lever, and the profile is meant to be the right per-mode setting at every `p_mix`.

4.1 What is NOT layered

The presets also set three booleans, and these still use the older `given()`-guarded assignment rather than `DbLayer`: `curated`, `curated_in_comp` (both `--gss`) and `db_advance` (both presets). They are not part of the specificity lattice — there is no “curated for COMP only” — so folding them in would have bought nothing. `--no-curated` and friends still apply first, so the preset blocks read settled values.

Twist knobs, the thermostat, and the `--profile` controller are untouched by this change.

knob	shipped	--phase-a	--gss
s_db (base)	9	—	6
p_convex (base)	0.4	—	0.5
p_mingen (base)	0.8	0.6	0.5
db_prefixes (base)	on	—	—
s_db_comp	12 [†]	—	12
s_db_comp_ctg	—	—	6
p_convex_comp	0.9 [†]	—	0.95
p_mingen_comp	—	0	0
db_prefixes_mix	—	—	off
db_prefixes_comp	—	—	on
s_db_ctg	—	—	—

Table 1: [†] withheld when --s-db / --p-convex is given explicitly. A dash means the layer has no opinion.

5 Worked examples

Taken verbatim from the DB effective per mode banner of e3cf2b54. pcv = p_convex, pmg = p_mingen, dsc = descent.

command line	MIX	COMP
(bare defaults)	pcv 0.4, 9/9, pmg 0.8, dsc on	pcv 0.9, 12/12, pmg 0.8, dsc on
--db-mode comp --s-db 20	pcv 0.4, 20/20 , pmg 0.8, dsc on	pcv 0.9, 20/20 , pmg 0.8, dsc on
--p-convex 1.0	pcv 1.0 , 9/9, pmg 0.8, dsc on	pcv 1.0 , 12/12, pmg 0.8, dsc on
...--s-db 20 --s-db-comp 7	pcv 0.4, 20/20, pmg 0.8, dsc on	pcv 0.9, 7/7 , pmg 0.8, dsc on
--gss	pcv 0.5, 6/6, pmg 0.5, dsc off	pcv 0.95, 12/6 , pmg 0, dsc on
--gss --s-db 15	pcv 0.5, 15/15 , pmg 0.5, dsc off	pcv 0.95, 12/6 , pmg 0, dsc on
--gss --s-db-comp 20	pcv 0.5, 6/6, pmg 0.5, dsc off	pcv 0.95, 20/6 , pmg 0, dsc on
--phase-a	pcv 0.4, 9/9, pmg 0.6 , dsc on	pcv 0.9, 12/12, pmg 0 , dsc on
--phase-a --gss	pcv 0.5, 6/6, pmg 0.5, dsc off	pcv 0.95, 12/6, pmg 0, dsc on
--no-db-prefixes	pcv 0.4, 9/9, pmg 0.8, dsc off	pcv 0.9, 12/12, pmg 0.8, dsc off
--p-mingen-comp 0	pcv 0.4, 9/9, pmg 0.8, dsc on	pcv 0.9, 12/12, pmg 0 , dsc on

Table 2: s_{db} shown as convex/contiguous.

Five of these repay a second look.

Row 2 is the bug that motivated the change: COMP now honours the explicit --s-db 20 instead of its own defaulted 12. **Row 3** is the same story for p_convex.

Row 4 shows the exception is narrow — an explicit mode knob still wins over an explicit base knob, because that is a specificity contest within one source, not a source contest.

Row 6 shows the preset asymmetry: --s-db 15 moves MIX but leaves the GSS profile’s COMP block intact.

Row 7 is the COMP-contiguous fallback in action: overriding --s-db-comp moves COMP convex to 20, while contiguous stays at the profile’s explicit 6 — because s_db_comp_ctg is set by the preset and is more specific.

Row 9 shows GSS sits above phase A where they overlap (p_mingen 0.5 not 0.6), and that phase A’s non-DB block still applies.

6 Reading a run

Every DB run prints one line:

```
[fmix] DB effective per mode:  MIX p_convex=0.5 s_db(cvx)=6 s_db(ctg)=6 p_mingen=0.5 descent=false |
                                COMP ...
```

This is produced by calling `MixParams::db_knobs` — the same function the mixer calls — so it is what runs, by construction. The previous banner echoed the base flags and separately re-derived the fall-through, which made it a second copy of the rules, free to drift. **When auditing a run, read this line, not the flags in the launch script.**

7 Errors

A knob that cannot possibly fire is now a hard error rather than a silent no-op. With the overlay off (`p_mix < 0`) only one mode ever runs, so the other mode’s overrides can never be read:

```
[fmix] ERROR: --s-db-comp can never take effect: --db-mode mix with no --p-mix overlay
          means COMP-DB rounds never happen. Drop the flag, or arm the overlay with --p-mix.
```

The message names only the flags actually typed — it reads the `cli` layer, not the settled values, since after composition every override holds something. Silently-inert flags are precisely how the shadowing bug survived two days.

8 Compatibility

This changes what some existing command lines do, deliberately.

- A script passing `--s-db N` with `--db-mode comp` used to get 12 and now gets `N`. Likewise `--p-convex` used to get 0.9 and now gets what it says.
- Scripts that passed both the base and the mode knob explicitly are unaffected.
- **Any calibration performed before 2026-08-05 that relied on a base knob in COMP mode was measuring something other than what its command line said.** Re-check before trusting it.
- A *resume* rebuilds its parameters from the command line, so a resumed run picks up the new rule. The banner makes this auditable.

9 Where to look

<code>src/bin/fmix.rs</code>	<code>DbLayer</code> , <code>DbLayer::shipped/gss/phase_a</code> , the <code>cli.over(...)</code> line, the inert-knob check, the effective-per-mode banner
<code>src/postmix/mix.rs</code>	<code>MixParams::db_knobs</code> , <code>ResolvedDbKnobs</code> , the <code>active_*</code> accessors
<code>docs/POSTMIX_MANUAL.md</code>	§2.1.2 (per-geometry, per-mode), §2.1.3 (the GSS profile)
<code>tests</code>	<code>db_knobs_layering_rules</code> , <code>s_db_resolves_by_mode_and_geometry</code> , <code>prefix_descent_resolves_per_mode</code>
